Evaluating Cryptographic API Misuse Detectors for Go

Vivi Andersson, Martin Monperrus

KTH Royal Institute of Technology, Stockholm, Sweden

April 18, 2026
Rio de Janeiro, Brazil
SVM 2026

Cryptographic API Misuse

Traefik, CVE-2025-66491. Kubernetes ingress provider:

```
InsecureSkipVerify: strings.ToLower(
    ptr.Deref(cfg.ProxySSLVerify, "off")) ==
        "on",
```

- The user sets `proxy-ssl-verify: "on"`, expecting TLS verification to be **enabled**.
- Expression evaluates to `true`, `InsecureSkipVerify` becomes `true`, inverting security assumptions.
- TLS encryption remains, but backend authentication is disabled: Traefik may trust a spoofed backend.

Evaluation Challenge

- Existing Go detectors do not operate over a common target: they differ in supported rules, reporting granularity, and execution robustness.
- Raw alert counts are therefore hard to compare: more findings may reflect broader rule coverage, noisier heuristics, or simply a different reporting unit.
- Tools may differ in coverage, granularity, or robustness. We need to understand what each tool detects, not just how much.

Prior Work

- Java already has comparative studies of crypto misuse detectors.
- Go also has detector work: *CryptoGo* introduced the first Go detector, and GOPHER extended that line of work.
- What is still missing is a systematic comparison of the Go detectors currently available in practice.

This paper

We provide the first comparative study of currently available Go crypto misuse detectors.

Full citations are in the paper.

Research Questions

- **RQ1.** What is the design space of state-of-the-art tools for detecting cryptographic API misuses in Go?
- **RQ2.** To what extent are crypto misuse detection tools for Go consistent with each other?

Study Design

RQ1 compares tool scope, supported rules, and detection approach.
RQ2 runs all tools on 328 security-relevant Go projects and compares execution coverage, findings, and overlap.

- Goal: establish what these tools detect, how they differ, and what follows for practice and evaluation.

RQ1: Taxonomy and Tool Support

Category	IDs	Rules
Cryptographic primitives	01–03	insecure algorithms; insecure PRNG; deprecated Go function
Key management	04–06	constant/predictable key; short key length; static/predictable IV
Password-based KDF	07–09	short salt; predictable salt; low iterations
Transport security	10–11	HTTP protocol; TLS/SSL issues
Secure Shell	12–13	insecure SSH suite; no host key validation
Token auth	14	no JWT verification

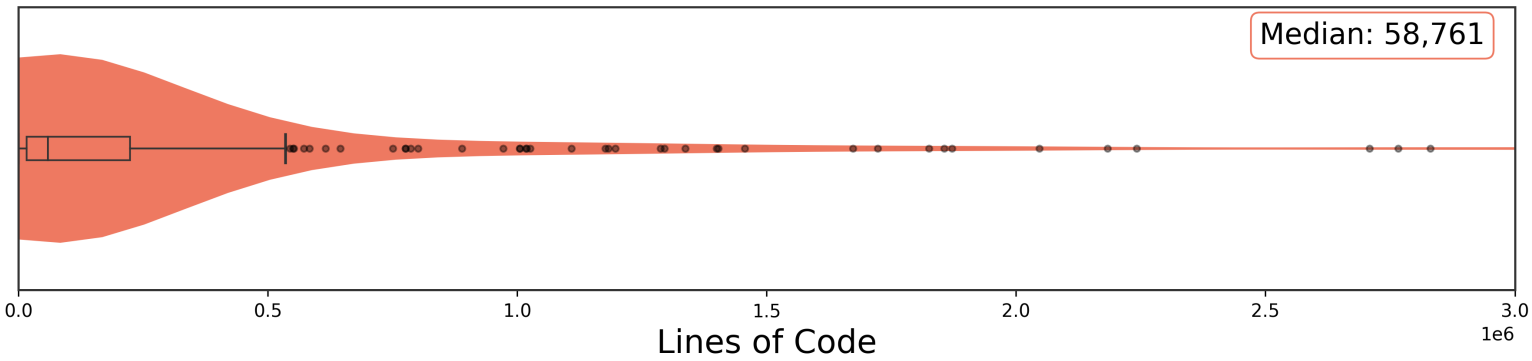
- Only rules 02, 05, and 11 are supported by all four tools.
- GOPHER covers the largest share of the taxonomy, while CODEQL alone supports rule 14.

RQ1: How the Tools Work

CodeQL	Datalog queries over a fact database, with data-flow reasoning.
Gopher	Taint tracking on SSA, with dynamically added rules.
Gosec	SSA for some rules, AST pattern matching for others.
Snyk Code	Proprietary static analysis; implementation details are undisclosed.

Dataset and Scope

- 328 security-relevant open-source Go repositories.
- Repository size is strongly right-skewed; the median is 58,761 LOC.
- Median popularity is 9,096 stars and median contributor count is 167.



RQ2: Execution Coverage Across Tools

	CodeQL	Gopher	Gosec	Snyk
Projects analyzed (%)	91.8	77.1	100	100
With findings (% of analyzed)	30.9	64.8	84.5	92.7
With findings (% of dataset)	28.4	50.0	84.5	92.7

Interpretation

Snyk Code and Gosec analyze all 328 projects, while Gopher fails on 75 and CodeQL on 27. Even before comparing findings, the tools operate on different subsets of the dataset.

RQ2: Tools Disagree on the Same Code

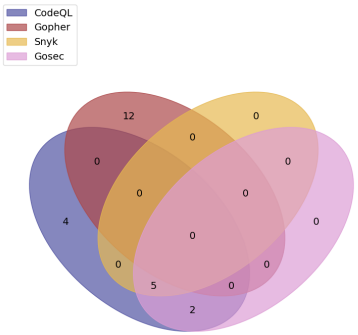
- 7,473 detections across 328 projects, collapsing to 6,152 unique locations.
- Only 1.3% of detections are shared by all four tools.
- 92.6% of Gosec detections and 63.5% of Snyk Code detections are unique to that tool.

Rule #02: insecure PRNG detection

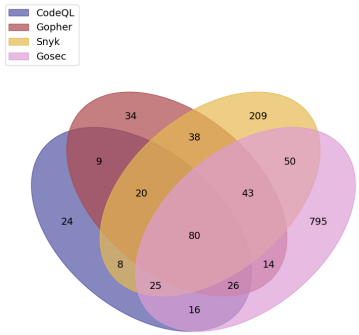
CodeQL: 14 Gopher: 0 Gosec: 2,517 Snyk Code: 17

RQ2: Agreement Depends on the Misuse Rule

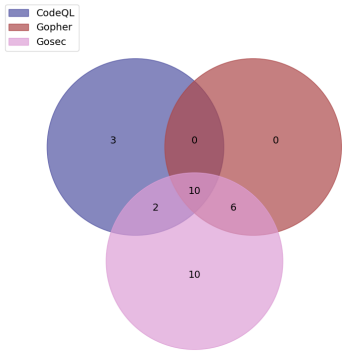
- Rule #05: all four tools support it, yet overlap remains limited.
- Rule #11: the largest class, with 2,033 detections total and 1,049 from Gosec alone.
- Rule #13: only 59 detections total, making manual inspection feasible.



Rule 05



Rule 11



Rule 13

RQ2 takeaway

Agreement depends strongly on the misuse class.

Manual Analysis: When Does Agreement Mean Truth?

- **Rule #05.** 4 of 5 detections shared by three tools (missed by GOPHER) are in mock or example code. Overlap is not always a correctness signal.
- **Rule #11.** 3 of 5 sampled GOSEC-only detections are false positives. SNYK CODE-only sample: no clear false positives.
- **Rule #13.** All 5 detections shared across tools are true positives. CODEQL uniquely catches custom callbacks beyond known patterns.

What Goes Wrong: Three Examples

1. Same call, different meaning *Rule #05, flagged by* GOPHER

```
h := blake2b.New512(nil)   // bug if used as a MAC
                           // fine for content hashing
```

Static analysis cannot tell which one this is.

2. Pattern match misses the equivalent *Rule #13, only* CODEQL

```
cfg.HostKeyCallback = func(...) error { return nil }
```

CODEQL detects this pattern; the other tools do not report it in our study.

3. Field name flagged, value ignored *Rule #11,* GOSEC *false positive*

```
tls.Config{MinVersion: tls.VersionTLS12}   // safe
```

In our manual sample, GOSEC flags this pattern even when the configured value is TLS 1.2.

Implications

- **For practitioners:** no single detector is enough. The tools miss different things, so relying on one leaves systematic blind spots.
- **For evaluators:** raw counts mislead. On insecure PRNG detection, GOSEC reports 2,517 findings where GOPHER reports 0, on the same code and for the same rule.
- **For tool builders:** the gap is qualitative, not only quantitative. The tools differ in rule coverage and in how much program context they model.

Closing claim

Detector choice changes both what practitioners see in code review and what researchers conclude in evaluation.

Future Work

- **API-level security invariants**. The paper points to formalization and enforcement of API-level security invariants as a research need.
- **More precise static reasoning**. Several disagreements come from limited context sensitivity around how values are configured and used.

Questions

Thank you

Questions and discussion